

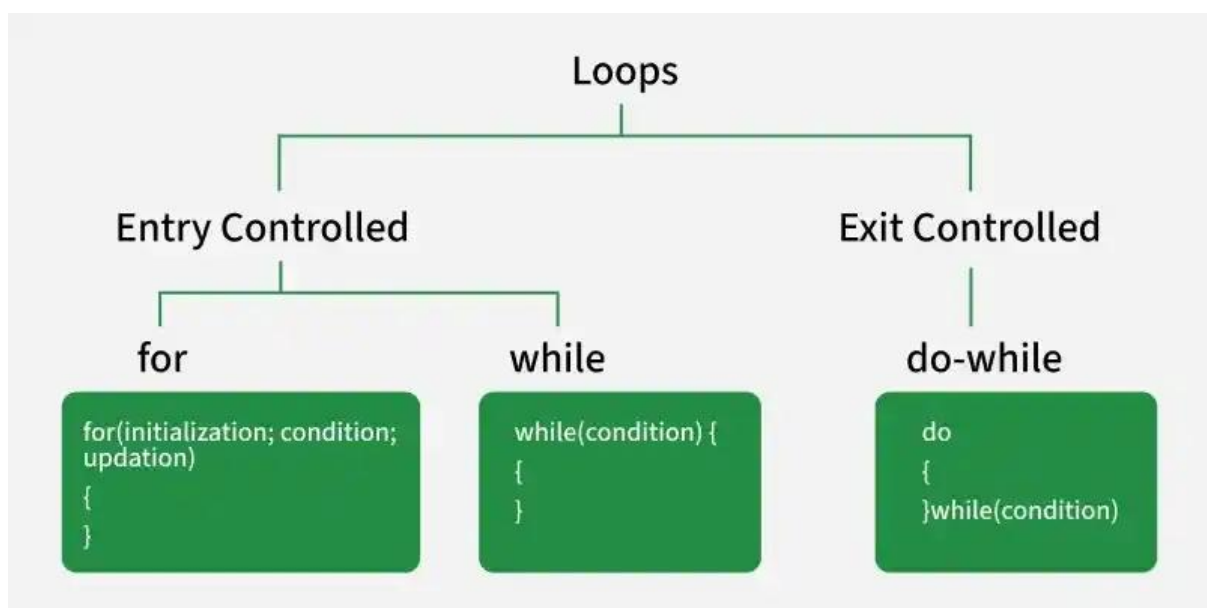
LOOPS

Loops in Java

Loops are control statements in Java that allow a block of code to be executed repeatedly as long as a specified condition is true. They help in reducing code repetition.

There are 4 types of loops in Java.

- **for:** Used when the number of iterations is known.
- **while:** Used when the number of iterations is not known in advance, condition checked before each iteration.
- **do-while:** Similar to while loop, but condition is checked after executing the block (executes at least once).
- **for-each:** Used to iterate over arrays and collections.



1. for loop

The for [loop](#) is used when we know the number of iterations (we know how many times we want to repeat a task). The for statement includes the initialization, condition, and increment/decrement in one line.

Example:

```
import java.io.*;

class Geeks {

    public static void main(String[] args)

    {

        for (int i = 0; i <= 10; i++) {

            System.out.print(i + " ");

        }

    }

}
```

Enhanced for loop (for each)

This loop is used to iterate over arrays or collections.

Example: The below Java program demonstrates an Enhanced for loop (for each loop) to iterate through an array and print names.

```
import java.io.*;
```

```
class Geeks {  
    public static void main(String[] args)  
    {  
        String[] names = { "Sweta", "Gudly", "Amiya" };  
  
        for (String name : names) {  
            System.out.println("Name: " + name);  
        }  
    }  
}
```

2. while Loop

A [while](#) loop is used when we want to check the condition before executing the loop body.

Example: Program demonstrates a while loop that prints numbers from 0 to 10 in a single line.

```
import java.io.*;
```

```
class Geeks {  
    public static void main(String[] args)  
    {  
        int i = 0;  
        while (i <= 10) {  
            System.out.print(i + " ");  
            i++;  
        }  
    }  
}
```

```
}  
}
```

Output

```
0 1 2 3 4 5 6 7 8 9 10
```

3. do-while Loop

The [do-while](#) loop ensures that the code block executes at least once before checking the condition.

Example: Program demonstrates a do-while loop that prints numbers from 0 to 10 in a single line.

```
import java.io.*;
```

```
class Geeks {  
    public static void main(String[] args)  
    {  
        int i = 0;  
        do {  
            System.out.print(i + " ");  
            i++;  
        } while (i <= 10);  
    }  
}
```

Output

```
0 1 2 3 4 5 6 7 8 9 10
```

Syntax

```
do {
```

```
// code to be executed
```

```
} while (condition);
```

Loop Type	When to Use	Condition Checking	Executes At Least Once?
for loop	When you want exact iterations	Before loop body, It is called Entry-controlled.	no
while loop	When you need condition check first.	Before loop body, It is called Entry-controlled.	no
do-while loop	When you need to run at least once	After loop body, It is called Exit-controlled.	yes
for-each loop	When you process all collection items	Internally handled	no

Break statements

Break and continue are jump statements used to alter the normal flow of loops. They help control loop execution by either terminating the loop early or skipping specific iterations. These statements can be used inside for, while, and do-while loops.

Break

The [break statement](#) in Java is used to immediately terminate a loop or switch statement and transfer control to the statement following it.

- Exits a loop before its condition becomes false
- Terminates execution when a specific condition is met
- Used to exit a switch case or break out of labeled (nested) loops

Syntax

break;

Example 1: Using break to exit a loop

Using break, we can force immediate termination of a loop, bypassing the conditional expression and any remaining code in the body of the loop. When we use break inside the nested loops, it will only break out of the innermost loop.

Example:

```
class GFG {  
  
    public static void main(String[] args) {  
  
        for (int i = 0; i < 10; i++) {
```

```
        if (i == 5)
            break;

        System.out.println("i: " + i);
    }
    System.out.println("Out of Loop");
}
```

Output

i: 0

i: 1

i: 2

i: 3

i: 4

Out of Loop

Explanation: The loop terminates as soon as i becomes 5, without checking further conditions.

Continue

The [continue statement](#) in Java is used to skip the current iteration of a loop and move directly to the next iteration.

- Skips specific values or conditions within a loop
- Avoids executing remaining statements in the current iteration
- Improves readability when filtering data in loops

Syntax

continue;

Example 1: Using continue to skip an iteration

```
class GFG {  
    public static void main(String args[]) {  
        for (int i = 0; i < 10; i++) {  
            if (i == 2)  
                continue;  
            System.out.print(i + " ");  
        }  
    }  
}
```

Output

0 1 3 4 5 6 7 8 9

Explanation: When i equals 2, the iteration is skipped and the loop continues with the next value.

Difference between break and continue:

Break	Continue
Terminates the loop immediately.	Skips the current iteration and continues with the next one
Transfers control outside the loop.	Transfers control to the next loop iteration.
Can be used with loops and switch statements.	Can be used only inside loops.
Stops further execution of the loop body completely.	Skips only the remaining statements in the current iteration.
Can be used with labeled loops.	Can also be used with labeled loops.